Java Stream API — Practice Workbook

Employee / EmployeeData exercises with solutions and expected output

75 questions across 9 sections

Before you start: a bug in the pasted code

The Employee constructor and the EmployeeData calls did not line up, so the original code will not compile. The constructor is declared as:

```
Employee(int id, String name, String gender, int age, String department,
         String city, double salary, int yearOfJoining, List<String> skills)
```

But the data was being passed as (id, name, department, gender, age, salary, LocalDate, city) — four separate problems:

1. department and gender are swapped, so gender would hold "IT" and department would hold "Male".
2. age and salary land in the wrong parameters (salary is a double, age is an int).
3. A LocalDate is passed where the class declares int yearOfJoining — that will not compile.
4. The skills list is never supplied, so the argument count is 8 instead of 9.

Also: Employee is declared package-private (class Employee) while EmployeeData is public, and the two files use different packages. Both classes below are in package practice and both are public. Corrected sources follow — use these for the exercises.

Setup: the two classes

Employee.java

```
package practice;

import java.util.List;

public class Employee {
    private int id;
    private String name;
    private String gender;
    private int age;
    private String department;
    private String city;
    private double salary;
    private int yearOfJoining;
    private List<String> skills;

    public Employee(int id, String name, String gender, int age, String department,
                    String city, double salary, int yearOfJoining, List<String> skills) {
        this.id = id;
        this.name = name;
        this.gender = gender;
        this.age = age;
        this.department = department;
```

```
        this.city = city;
        this.salary = salary;
        this.yearOfJoining = yearOfJoining;
        this.skills = skills;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getGender() { return gender; }
    public int getAge() { return age; }
    public String getDepartment() { return department; }
    public String getCity() { return city; }
    public double getSalary() { return salary; }
    public int getYearOfJoining() { return yearOfJoining; }
    public List<String> getSkills() { return skills; }

    @Override
    public String toString() {
        return "Employee [id=" + id + ", name=" + name + ", gender=" + gender
                + ", age=" + age + ", department=" + department + ", city=" + city
                + ", salary=" + salary + ", yearOfJoining=" + yearOfJoining
                + ", skills=" + skills + "]";
    }
}
```

EmployeeData.java

```
package practice;

import java.util.Arrays;
import java.util.List;

public class EmployeeData {

    private EmployeeData() {
        // utility class, no instances
    }

    // (id, name, gender, age, department, city, salary, yearOfJoining, skills)
    public static List<Employee> getEmployees() {
        return Arrays.asList(
    new Employee(101, "Aarav Mehta", "Male", 28, "IT", "Pune",
            75000, 2019, Arrays.asList("Java", "Spring", "SQL")),
    new Employee(102, "Priya Sharma", "Female", 34, "HR", "Delhi",
            55000, 2015, Arrays.asList("Recruitment", "Excel")),
    new Employee(103, "Rohit Verma", "Male", 41, "IT", "Bangalore",
            125000, 2012, Arrays.asList("Java", "Microservices", "AWS", "Kafka")),
    new Employee(104, "Sneha Kulkarni", "Female", 30, "Finance", "Mumbai",
            68000, 2018, Arrays.asList("Excel", "Tally", "SQL")),
    new Employee(105, "Imran Khan", "Male", 25, "IT", "Pune",
            48000, 2022, Arrays.asList("Java", "SQL")),
    new Employee(106, "Neha Gupta", "Female", 38, "Marketing", "Delhi",
            92000, 2014, Arrays.asList("SEO", "Content", "Analytics")),
    new Employee(107, "Vikram Rao", "Male", 45, "Finance", "Bangalore",
            140000, 2010, Arrays.asList("Excel", "SAP", "Audit")),
    new Employee(108, "Anjali Nair", "Female", 32, "IT", "Chennai",
            88000, 2017, Arrays.asList("Python", "SQL", "AWS")),
    new Employee(109, "Karan Singh", "Male", 29, "Sales", "Jaipur",
            52000, 2020, Arrays.asList("Negotiation", "CRM")),
    new Employee(110, "Meera Iyer", "Female", 27, "HR", "Chennai",
            46000, 2021, Arrays.asList("Payroll", "Excel")),
    new Employee(111, "Suresh Patil", "Male", 50, "Sales", "Pune",
            99000, 2008, Arrays.asList("Negotiation", "CRM", "Leadership")),
    new Employee(112, "Divya Menon", "Female", 31, "Marketing", "Mumbai",
            71000, 2019, Arrays.asList("SEO", "Analytics")),
    new Employee(113, "Arjun Reddy", "Male", 36, "IT", "Hyderabad",
            110000, 2013, Arrays.asList("Java", "Spring", "Kafka", "Docker")),
    new Employee(114, "Fatima Sheikh", "Female", 43, "Finance", "Hyderabad",
            118000, 2011, Arrays.asList("SAP", "Audit", "Excel")),
    new Employee(115, "Manish Joshi", "Male", 24, "Sales", "Jaipur",
            40000, 2023, Arrays.asList("CRM"))
        );
    }
}
```

Boilerplate for your practice class

```
package practice;
```

```
import java.util.*;
import java.util.stream.*;

public class StreamPractice {
    public static void main(String[] args) {
        List<Employee> employees = EmployeeData.getEmployees();
        // paste any solution here
    }
}
```

Every solution in this document assumes a local variable named employees holding EmployeeData.getEmployees().

The dataset at a glance

Keep this page open while practising — it lets you verify answers by eye.

ID	Name	Gen	Age	Dept	City	Salary	Yr	Skills
101	Aarav Mehta	Male	28	IT	Pune	75000	2019	Java, Spring, SQL
102	Priya Sharma	Female	34	HR	Delhi	55000	2015	Recruitment, Excel
103	Rohit Verma	Male	41	IT	Bangalore	125000	2012	Java, Microservices, AWS, Kafka
104	Sneha Kulkarni	Female	30	Finance	Mumbai	68000	2018	Excel, Tally, SQL
105	Imran Khan	Male	25	IT	Pune	48000	2022	Java, SQL
106	Neha Gupta	Female	38	Marketing	Delhi	92000	2014	SEO, Content, Analytics
107	Vikram Rao	Male	45	Finance	Bangalore	140000	2010	Excel, SAP, Audit
108	Anjali Nair	Female	32	IT	Chennai	88000	2017	Python, SQL, AWS
109	Karan Singh	Male	29	Sales	Jaipur	52000	2020	Negotiation, CRM
110	Meera Iyer	Female	27	HR	Chennai	46000	2021	Payroll, Excel
111	Suresh Patil	Male	50	Sales	Pune	99000	2008	Negotiation, CRM, Leadership
112	Divya Menon	Female	31	Marketing	Mumbai	71000	2019	SEO, Analytics
113	Arjun Reddy	Male	36	IT	Hyderabad	110000	2013	Java, Spring, Kafka, Docker
114	Fatima Sheikh	Female	43	Finance	Hyderabad	118000	2011	SAP, Audit, Excel
115	Manish Joshi	Male	24	Sales	Jaipur	40000	2023	CRM

15 employees · 5 departments (IT 5, Finance 3, Sales 3, HR 2, Marketing 2) · 8 male, 7 female · 7 cities · salaries 40,000 to 140,000 · total payroll 1,227,000.

Part 1 — Questions only (attempt these first)

Try to write each one without looking at Part 2. Solutions start on the following pages.

Section A — filter, map, distinct (warm-up)

Q1. Print all employees.
Q2. Get the names of all employees.
Q3. Get the names of all employees in the IT department.
Q4. Find all employees earning more than 80000.
Q5. Find all female employees working in IT (two conditions).
Q6. Get all distinct cities.
Q7. Get all distinct departments.
Q8. Find employees whose age is between 25 and 35 (inclusive).
Q9. Find employees based in Pune or Delhi.
Q10. Print IT employee names in uppercase.

Section B — sorting, limit, skip

Q11. Sort employees by salary in ascending order.
Q12. Sort employees by salary in descending order.
Q13. Sort employee names alphabetically.
Q14. Sort by department ascending, then by salary descending (chained comparator).
Q15. Find the top 3 highest paid employees.
Q16. Find the 3 youngest employees.
Q17. Skip the first 5 employees and take the next 3 (pagination pattern).
Q18. Sort employees by age descending.

Section C — min, max and Optional

Q19. Find the highest paid employee.
Q20. Find the lowest paid employee.
Q21. Find the youngest employee.
Q22. Find the oldest employee.
Q23. Find the youngest male employee.
Q24. Find the oldest female employee.
Q25. Find the most senior employee (earliest year of joining).
Q26. Find the highest paid employee in the IT department.

Section D — aggregation and statistics

Q27. Find the total salary paid by the company.
Q28. Find the average salary of all employees.
Q29. Find the average age of all employees.

Q30. Count the total number of employees.
Q31. Get full salary statistics in one pass (count, sum, min, average, max).
Q32. Find the maximum and minimum age.
Q33. Find the sum of all ages using reduce.
Q34. Find the average salary of male and female employees separately.

Section E — groupingBy (the most asked category)

Q35. Count the number of employees in each department.
Q36. Group employee names by department.
Q37. Find the average salary of each department.
Q38. Find the total salary of each department.
Q39. Count male and female employees.
Q40. Count employees per city, then find the city with the most employees.
Q41. Group employees by year of joining, keeping the years in sorted order.
Q42. Find the department with the highest average salary.
Q43. Find the highest paid employee in each department (collectingAndThen).
Q44. Count employees by department and gender (nested groupingBy).
Q45. Find departments that have more than 2 employees.
Q46. List departments sorted by total salary, highest first (preserve order).

Section F — partitioningBy

Q47. Partition employees into those earning more than 80000 and the rest.
Q48. Partition employees by age above 35.
Q49. Count how many employees are in IT versus not in IT.

Section G — flatMap on the skills list

Q50. Get all distinct skills across the company.
Q51. Count how many employees have each skill.
Q52. Find all employees who know Java.
Q53. Find the most common skill in the company.
Q54. Get the distinct set of skills present in each department.
Q55. Find employees who have 3 or more skills.
Q56. Find the total number of skill entries (not distinct) in the company.

Section H — toMap, joining, matching

Q57. Build a Map of employee id to employee name.
Q58. Build a Map of employee name to salary.
Q59. Build a Map of department to a comma separated string of names (merge function).
Q60. Join all employee names into a single comma separated string with brackets.
Q61. Is there any employee older than 48?
Q62. Do all employees earn more than 35000?

Q63. Is there no employee from Kolkata?

Section I — interview favourites (tricky)

Q64. Find the second highest salary.
Q65. Find the Nth highest salary (write it as a reusable method).
Q66. Find the second highest salary within the IT department.
Q67. Give every employee a 10% raise without mutating the original objects.
Q68. Find the first employee earning more than 100000 (short-circuiting).
Q69. Find employees with more than 10 years of experience (assume current year 2026).
Q70. Group employee names by the first letter of their name.
Q71. Collect names and salaries into a LinkedHashMap ordered by salary descending.
Q72. Find names of employees starting with 'A' or 'S'.
Q73. Find the average age of employees in the IT department.
Q74. Print the employee count per department, sorted by count descending then department name.
Q75. Using reduce, find the employee with the highest salary (without max()).

Part 2 — Solutions with expected output

Note on Map ordering: groupingBy and toMap return a HashMap, so the printed order can differ from what is shown here. Only the key/value pairs matter. Where order is part of the answer, the solution explicitly asks for TreeMap or LinkedHashMap.

Section A — filter, map, distinct (warm-up)

Q1. Print all employees.

```
employees.forEach(System.out::println);
```

Output: `Employee [id=101, name=Aarav Mehta, ...]  (all 15 rows)`

Q2. Get the names of all employees.

```
List<String> names = employees.stream()
        .map(Employee::getName)
        .collect(Collectors.toList());
```

Output: `[Aarav Mehta, Priya Sharma, Rohit Verma, ... Manish Joshi]`

Q3. Get the names of all employees in the IT department.

```
employees.stream()
        .filter(e -> "IT".equals(e.getDepartment()))
        .map(Employee::getName)
        .collect(Collectors.toList());
```

Output: `[Aarav Mehta, Rohit Verma, Imran Khan, Anjali Nair, Arjun Reddy]`

Q4. Find all employees earning more than 80000.

```
employees.stream()
        .filter(e -> e.getSalary() > 80000)
        .map(Employee::getName)
        .collect(Collectors.toList());
```

Output: `[Rohit Verma, Neha Gupta, Vikram Rao, Anjali Nair, Suresh Patil, Arjun Reddy, Fatima Sheikh]`

Q5. Find all female employees working in IT (two conditions).

```
employees.stream()
        .filter(e -> "Female".equals(e.getGender()))
        .filter(e -> "IT".equals(e.getDepartment()))
        .map(Employee::getName)
        .collect(Collectors.toList());
```

Output: `[Anjali Nair]`

Q6. Get all distinct cities.

```
employees.stream()
        .map(Employee::getCity)
        .distinct()
        .sorted()
        .collect(Collectors.toList());
```

Output: [Bangalore, Chennai, Delhi, Hyderabad, Jaipur, Mumbai, Pune]

Q7. Get all distinct departments.

```
employees.stream()
        .map(Employee::getDepartment)
        .distinct()
        .sorted()
        .collect(Collectors.toList());
```

Output: [Finance, HR, IT, Marketing, Sales]

Q8. Find employees whose age is between 25 and 35 (inclusive).

```
employees.stream()
        .filter(e -> e.getAge() >= 25 && e.getAge() <= 35)
        .map(e -> e.getName() + " (" + e.getAge() + ")")
        .collect(Collectors.toList());
```

Output: [Aarav Mehta (28), Priya Sharma (34), Sneha Kulkarni (30), Imran Khan (25), Anjali Nair (32), Karan Singh (29), Meera Iyer (27), Divya Menon (31)]

Q9. Find employees based in Pune or Delhi.

```
List<String> targets = List.of("Pune", "Delhi");
employees.stream()
        .filter(e -> targets.contains(e.getCity()))
        .map(Employee::getName)
        .collect(Collectors.toList());
```

Output: [Aarav Mehta, Priya Sharma, Imran Khan, Neha Gupta, Suresh Patil]

Q10. Print IT employee names in uppercase.

```
employees.stream()
        .filter(e -> "IT".equals(e.getDepartment()))
        .map(Employee::getName)
        .map(String::toUpperCase)
        .collect(Collectors.toList());
```

Output: [AARAV MEHTA, ROHIT VERMA, IMRAN KHAN, ANJALI NAIR, ARJUN REDDY]

Section B — sorting, limit, skip

Q11. Sort employees by salary in ascending order.

```
employees.stream()
        .sorted(Comparator.comparingDouble(Employee::getSalary))
        .map(Employee::getName)
        .collect(Collectors.toList());
```

Output: [Manish Joshi, Meera Iyer, Imran Khan, Karan Singh, Priya Sharma, Sneha Kulkarni, Divya Menon, Aarav Mehta, Anjali Nair, Neha Gupta, Suresh Patil, Arjun Reddy, Fatima Sheikh, Rohit Verma, Vikram Rao]

Q12. Sort employees by salary in descending order.

```
employees.stream()
        .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
        .map(Employee::getName)
        .collect(Collectors.toList());
```

Output: [Vikram Rao, Rohit Verma, Fatima Sheikh, Arjun Reddy, Suresh Patil, Neha Gupta, Anjali Nair, Aarav Mehta, Divya Menon, Sneha Kulkarni, Priya Sharma, Karan Singh, Imran Khan, Meera Iyer, Manish Joshi]

Q13. Sort employee names alphabetically.

```
employees.stream()
        .map(Employee::getName)
        .sorted()
        .collect(Collectors.toList());
```

Output: [Aarav Mehta, Anjali Nair, Arjun Reddy, Divya Menon, Fatima Sheikh, Imran Khan, Karan Singh, Manish Joshi, Meera Iyer, Neha Gupta, Priya Sharma, Rohit Verma, Sneha Kulkarni, Suresh Patil, Vikram Rao]

Q14. Sort by department ascending, then by salary descending (chained comparator).

```
employees.stream()
        .sorted(Comparator.comparing(Employee::getDepartment)
                .thenComparing(Comparator.comparingDouble(Employee::getSalary)
                        .reversed()))
        .map(e -> e.getDepartment() + " | " + e.getName() + " | " + e.getSalary())
        .forEach(System.out::println);
```

Output:
```
Finance | Vikram Rao | 140000.0
Finance | Fatima Sheikh | 118000.0
Finance | Sneha Kulkarni | 68000.0
HR | Priya Sharma | 55000.0
HR | Meera Iyer | 46000.0
IT | Rohit Verma | 125000.0
IT | Arjun Reddy | 110000.0
IT | Anjali Nair | 88000.0
IT | Aarav Mehta | 75000.0
IT | Imran Khan | 48000.0
Marketing | Neha Gupta | 92000.0
Marketing | Divya Menon | 71000.0
Sales | Suresh Patil | 99000.0
Sales | Karan Singh | 52000.0
Sales | Manish Joshi | 40000.0
```

Q15. Find the top 3 highest paid employees.

```
employees.stream()
        .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
        .limit(3)
        .map(e -> e.getName() + " - " + e.getSalary())
        .collect(Collectors.toList());
```

Output: [Vikram Rao - 140000.0, Rohit Verma - 125000.0, Fatima Sheikh - 118000.0]

Q16. Find the 3 youngest employees.

```
employees.stream()
        .sorted(Comparator.comparingInt(Employee::getAge))
        .limit(3)
        .map(e -> e.getName() + " (" + e.getAge() + ")")
        .collect(Collectors.toList());
```

Output: [Manish Joshi (24), Imran Khan (25), Meera Iyer (27)]

Q17. Skip the first 5 employees and take the next 3 (pagination pattern).

```
employees.stream()
        .skip(5)
        .limit(3)
        .map(Employee::getName)
        .collect(Collectors.toList());
```

Output: `[Neha Gupta, Vikram Rao, Anjali Nair]`

Q18. Sort employees by age descending.

```
employees.stream()
        .sorted(Comparator.comparingInt(Employee::getAge).reversed())
        .map(e -> e.getName() + ":" + e.getAge())
        .collect(Collectors.toList());
```

Output: `[Suresh Patil:50, Vikram Rao:45, Fatima Sheikh:43, Rohit Verma:41, Neha Gupta:38, Arjun Reddy:36, Priya Sharma:34, Anjali Nair:32, Divya Menon:31, Sneha Kulkarni:30, Karan Singh:29, Aarav Mehta:28, Meera Iyer:27, Imran Khan:25, Manish Joshi:24]`

Section C — min, max and Optional

Q19. Find the highest paid employee.

```
Optional<Employee> top = employees.stream()
        .max(Comparator.comparingDouble(Employee::getSalary));
top.map(Employee::getName).orElse("N/A");
```

Output: `Vikram Rao`

Q20. Find the lowest paid employee.

```
employees.stream()
        .min(Comparator.comparingDouble(Employee::getSalary))
        .map(Employee::getName)
        .orElse("N/A");
```

Output: `Manish Joshi`

Q21. Find the youngest employee.

```
employees.stream()
        .min(Comparator.comparingInt(Employee::getAge))
        .map(Employee::getName)
        .orElse("N/A");
```

Output: `Manish Joshi`

Q22. Find the oldest employee.

```
employees.stream()
        .max(Comparator.comparingInt(Employee::getAge))
        .map(Employee::getName)
        .orElse("N/A");
```

Output: `Suresh Patil`

Q23. Find the youngest male employee.

```
employees.stream()
        .filter(e -> "Male".equals(e.getGender()))
        .min(Comparator.comparingInt(Employee::getAge))
```

```
        .map(Employee::getName)
        .orElse("N/A");
```

Output: `Manish Joshi`

Q24. Find the oldest female employee.

```
employees.stream()
        .filter(e -> "Female".equals(e.getGender()))
        .max(Comparator.comparingInt(Employee::getAge))
        .map(Employee::getName)
        .orElse("N/A");
```

Output: `Fatima Sheikh`

Q25. Find the most senior employee (earliest year of joining).

```
employees.stream()
        .min(Comparator.comparingInt(Employee::getYearOfJoining))
        .map(e -> e.getName() + " (" + e.getYearOfJoining() + ")")
        .orElse("N/A");
```

Output: `Suresh Patil (2008)`

Q26. Find the highest paid employee in the IT department.

```
employees.stream()
        .filter(e -> "IT".equals(e.getDepartment()))
        .max(Comparator.comparingDouble(Employee::getSalary))
        .map(Employee::getName)
        .orElse("N/A");
```

Output: `Rohit Verma`

Section D — aggregation and statistics

Q27. Find the total salary paid by the company.

```
double total = employees.stream()
        .mapToDouble(Employee::getSalary)
        .sum();
```

Output: `1227000.0`

Q28. Find the average salary of all employees.

```
employees.stream()
        .mapToDouble(Employee::getSalary)
        .average()
        .orElse(0.0);
```

Output: `81800.0`

Q29. Find the average age of all employees.

```
employees.stream()
        .mapToInt(Employee::getAge)
        .average()
        .orElse(0.0);
```

Output: `34.2`

Q30. Count the total number of employees.

```
long count = employees.stream().count();
```

Output: 15

Q31. Get full salary statistics in one pass (count, sum, min, average, max).

```
DoubleSummaryStatistics stats = employees.stream()
        .mapToDouble(Employee::getSalary)
        .summaryStatistics();
System.out.println(stats);
```

Output: DoubleSummaryStatistics{count=15, sum=1227000.000000, min=40000.000000, average=81800.000000, max=140000.000000}

Q32. Find the maximum and minimum age.

```
IntSummaryStatistics ageStats = employees.stream()
        .mapToInt(Employee::getAge)
        .summaryStatistics();
ageStats.getMax();  // 50
ageStats.getMin();  // 24
```

Output: max = 50, min = 24

Q33. Find the sum of all ages using reduce.

```
int sumAges = employees.stream()
        .map(Employee::getAge)
        .reduce(0, Integer::sum);
```

Output: 513

Q34. Find the average salary of male and female employees separately.

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getGender,
                Collectors.averagingDouble(Employee::getSalary)));
```

Output: {Male=86125.0, Female=76857.142857142855}

Section E — groupingBy (the most asked category)

Q35. Count the number of employees in each department.

```
Map<String, Long> byDept = employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
                Collectors.counting()));
```

Output: {Finance=3, HR=2, IT=5, Marketing=2, Sales=3}

Q36. Group employee names by department.

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
                Collectors.mapping(Employee::getName, Collectors.toList())));
```

Output: {Finance=[Sneha Kulkarni, Vikram Rao, Fatima Sheikh],
HR=[Priya Sharma, Meera Iyer],
IT=[Aarav Mehta, Rohit Verma, Imran Khan, Anjali Nair, Arjun Reddy],
Marketing=[Neha Gupta, Divya Menon],
Sales=[Karan Singh, Suresh Patil, Manish Joshi]}

Q37. Find the average salary of each department.

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
                Collectors.averagingDouble(Employee::getSalary)));
```

Output: {Finance=108666.66666666667, HR=50500.0, IT=89200.0, Marketing=81500.0, Sales=63666.666666666664}

Q38. Find the total salary of each department.

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
                Collectors.summingDouble(Employee::getSalary)));
```

Output: {Finance=326000.0, HR=101000.0, IT=446000.0, Marketing=163000.0, Sales=191000.0}

Q39. Count male and female employees.

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getGender,
                Collectors.counting()));
```

Output: {Male=8, Female=7}

Q40. Count employees per city, then find the city with the most employees.

```
Map<String, Long> byCity = employees.stream()
        .collect(Collectors.groupingBy(Employee::getCity, Collectors.counting()));

byCity.entrySet().stream()
        .max(Map.Entry.comparingByValue())
        .map(e -> e.getKey() + " = " + e.getValue())
        .orElse("N/A");
```

Output: {Pune=3, Delhi=2, Bangalore=2, Mumbai=2, Chennai=2, Jaipur=2, Hyderabad=2}
Pune = 3

Q41. Group employees by year of joining, keeping the years in sorted order.

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getYearOfJoining,
                TreeMap::new,
                Collectors.mapping(Employee::getName, Collectors.toList())));
```

Output: {2008=[Suresh Patil], 2010=[Vikram Rao], 2011=[Fatima Sheikh],
2012=[Rohit Verma], 2013=[Arjun Reddy], 2014=[Neha Gupta],
2015=[Priya Sharma], 2017=[Anjali Nair], 2018=[Sneha Kulkarni],
2019=[Aarav Mehta, Divya Menon], 2020=[Karan Singh],
2021=[Meera Iyer], 2022=[Imran Khan], 2023=[Manish Joshi]}

Q42. Find the department with the highest average salary.

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
                Collectors.averagingDouble(Employee::getSalary)))
        .entrySet().stream()
        .max(Map.Entry.comparingByValue())
        .map(e -> e.getKey() + " = " + e.getValue())
        .orElse("N/A");
```

Output: Finance = 108666.66666666667

Q43. Find the highest paid employee in each department (collectingAndThen).

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
                Collectors.collectingAndThen(
                    Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary)),
                    opt -> opt.map(Employee::getName).orElse("N/A"))));
```

Output: `{Finance=Vikram Rao, HR=Priya Sharma, IT=Rohit Verma, Marketing=Neha Gupta, Sales=Suresh Patil}`

Q44. Count employees by department and gender (nested groupingBy).

```
Map<String, Map<String, Long>> nested = employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
                Collectors.groupingBy(Employee::getGender, Collectors.counting())));
```

Output: `{Finance={Male=1, Female=2}, HR={Female=2}, IT={Male=4, Female=1}, Marketing={Female=2}, Sales={Male=3}}`

Q45. Find departments that have more than 2 employees.

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment, Collectors.counting()))
        .entrySet().stream()
        .filter(e -> e.getValue() > 2)
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
```

Output: `{Finance=3, IT=5, Sales=3}`

Q46. List departments sorted by total salary, highest first (preserve order).

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
                Collectors.summingDouble(Employee::getSalary)))
        .entrySet().stream()
        .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                (a, b) -> a, LinkedHashMap::new));
```

Output: `{IT=446000.0, Finance=326000.0, Sales=191000.0, Marketing=163000.0, HR=101000.0}`

Section F — partitioningBy

Q47. Partition employees into those earning more than 80000 and the rest.

```
Map<Boolean, List<String>> parts = employees.stream()
        .collect(Collectors.partitioningBy(e -> e.getSalary() > 80000,
                Collectors.mapping(Employee::getName, Collectors.toList())));
```

Output: `{false=[Aarav Mehta, Priya Sharma, Sneha Kulkarni, Imran Khan, Karan Singh, Meera Iyer, Divya Menon, Manish Joshi],`
` true=[Rohit Verma, Neha Gupta, Vikram Rao, Anjali Nair, Suresh Patil, Arjun Reddy, Fatima Sheikh]}`

Q48. Partition employees by age above 35.

```
employees.stream()
        .collect(Collectors.partitioningBy(e -> e.getAge() > 35,
                Collectors.mapping(Employee::getName, Collectors.toList())));
```

Output: `{false=[Aarav Mehta, Priya Sharma, Sneha Kulkarni, Imran Khan, Anjali Nair, Karan Singh,`

```
Meera Iyer, Divya Menon, Manish Joshi],
 true=[Rohit Verma, Neha Gupta, Vikram Rao, Suresh Patil, Arjun Reddy, Fatima Sheikh]}
```

Q49. Count how many employees are in IT versus not in IT.

```
employees.stream()
        .collect(Collectors.partitioningBy(e -> "IT".equals(e.getDepartment()),
                Collectors.counting()));
```

Output: `{false=10, true=5}`

Section G — flatMap on the skills list

Q50. Get all distinct skills across the company.

```
employees.stream()
        .flatMap(e -> e.getSkills().stream())
        .distinct()
        .sorted()
        .collect(Collectors.toList());
```

Output: `[AWS, Analytics, Audit, CRM, Content, Docker, Excel, Java, Kafka, Leadership, Microservices, Negotiation, Payroll, Python, Recruitment, SAP, SEO, SQL, Spring, Tally]`

Q51. Count how many employees have each skill.

```
employees.stream()
        .flatMap(e -> e.getSkills().stream())
        .collect(Collectors.groupingBy(s -> s, Collectors.counting()));
```

Output: `{Excel=5, Java=4, SQL=4, CRM=3, Spring=2, AWS=2, Kafka=2, SEO=2, Analytics=2, SAP=2, Audit=2, Negotiation=2, Recruitment=1, Microservices=1, Tally=1, Content=1, Python=1, Payroll=1, Leadership=1, Docker=1}`

Q52. Find all employees who know Java.

```
employees.stream()
        .filter(e -> e.getSkills().contains("Java"))
        .map(Employee::getName)
        .collect(Collectors.toList());
```

Output: `[Aarav Mehta, Rohit Verma, Imran Khan, Arjun Reddy]`

Q53. Find the most common skill in the company.

```
employees.stream()
        .flatMap(e -> e.getSkills().stream())
        .collect(Collectors.groupingBy(s -> s, Collectors.counting()))
        .entrySet().stream()
        .max(Map.Entry.comparingByValue())
        .map(e -> e.getKey() + " (" + e.getValue() + ")")
        .orElse("N/A");
```

Output: `Excel (5)`

Q54. Get the distinct set of skills present in each department.

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
                Collectors.flatMapping(e -> e.getSkills().stream(),
                        Collectors.toCollection(TreeSet::new))));
```

Output:
```
{Finance=[Audit, Excel, SAP, SQL, Tally],
 HR=[Excel, Payroll, Recruitment],
 IT=[AWS, Docker, Java, Kafka, Microservices, Python, SQL, Spring],
 Marketing=[Analytics, Content, SEO],
 Sales=[CRM, Leadership, Negotiation]}
```

Q55. Find employees who have 3 or more skills.

```
employees.stream()
        .filter(e -> e.getSkills().size() >= 3)
        .map(e -> e.getName() + " -> " + e.getSkills().size())
        .collect(Collectors.toList());
```

Output:
```
[Aarav Mehta -> 3, Rohit Verma -> 4, Sneha Kulkarni -> 3, Neha Gupta -> 3, Vikram Rao -> 3, Anjali Nair -> 3, Suresh Patil -> 3, Arjun Reddy -> 4, Fatima Sheikh -> 3]
```

Q56. Find the total number of skill entries (not distinct) in the company.

```
long totalSkills = employees.stream()
        .flatMap(e -> e.getSkills().stream())
        .count();
```

Output:
```
40
```

Section H — toMap, joining, matching

Q57. Build a Map of employee id to employee name.

```
Map<Integer, String> idToName = employees.stream()
        .collect(Collectors.toMap(Employee::getId, Employee::getName));
```

Output:
```
{101=Aarav Mehta, 102=Priya Sharma, 103=Rohit Verma, ... 115=Manish Joshi}
```

Q58. Build a Map of employee name to salary.

```
Map<String, Double> nameToSalary = employees.stream()
        .collect(Collectors.toMap(Employee::getName, Employee::getSalary));
```

Output:
```
{Aarav Mehta=75000.0, Priya Sharma=55000.0, ... Manish Joshi=40000.0}
```

Q59. Build a Map of department to a comma separated string of names (merge function).

```
employees.stream()
        .collect(Collectors.toMap(Employee::getDepartment,
                Employee::getName,
                (a, b) -> a + ", " + b));
```

Output:
```
{Finance=Sneha Kulkarni, Vikram Rao, Fatima Sheikh,
 HR=Priya Sharma, Meera Iyer,
 IT=Aarav Mehta, Rohit Verma, Imran Khan, Anjali Nair, Arjun Reddy,
 Marketing=Neha Gupta, Divya Menon,
 Sales=Karan Singh, Suresh Patil, Manish Joshi}
```

Q60. Join all employee names into a single comma separated string with brackets.

```
employees.stream()
        .map(Employee::getName)
        .collect(Collectors.joining(", ", "[", "]"));
```

Output:
```
[Aarav Mehta, Priya Sharma, Rohit Verma, Sneha Kulkarni, Imran Khan, Neha Gupta, Vikram Rao, Anjali Nair, Karan Singh, Meera Iyer, Suresh Patil, Divya Menon, Arjun Reddy, Fatima Sheikh, Manish Joshi]
```

Q61. Is there any employee older than 48?

```
boolean any = employees.stream().anyMatch(e -> e.getAge() > 48);
```

Output: `true`

Q62. Do all employees earn more than 35000?

```
boolean all = employees.stream().allMatch(e -> e.getSalary() > 35000);
```

Output: `true`

Q63. Is there no employee from Kolkata?

```
boolean none = employees.stream()
        .noneMatch(e -> "Kolkata".equals(e.getCity()));
```

Output: `true`

Section I — interview favourites (tricky)

Q64. Find the second highest salary.

```
employees.stream()
        .map(Employee::getSalary)
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst()
        .orElse(0.0);
```

Output: `125000.0`

Q65. Find the Nth highest salary (write it as a reusable method).

```
static double nthHighestSalary(List<Employee> list, int n) {
    return list.stream()
            .map(Employee::getSalary)
            .distinct()
            .sorted(Comparator.reverseOrder())
            .skip(n - 1L)
            .findFirst()
            .orElse(0.0);
}

nthHighestSalary(employees, 3);
```

Output: `118000.0`

Q66. Find the second highest salary within the IT department.

```
employees.stream()
        .filter(e -> "IT".equals(e.getDepartment()))
        .map(Employee::getSalary)
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst()
        .orElse(0.0);
```

Output: `110000.0`

Q67. Give every employee a 10% raise without mutating the original objects.

```
List<Employee> raised = employees.stream()
        .map(e -> new Employee(e.getId(), e.getName(), e.getGender(), e.getAge(),
                e.getDepartment(), e.getCity(), e.getSalary() * 1.10,
                e.getYearOfJoining(), e.getSkills()))
        .collect(Collectors.toList());
```

Output: `Aarav Mehta -> 82500.0, Priya Sharma -> 60500.0, Rohit Verma -> 137500.0, ... Manish Joshi -> 44000.0`

Q68. Find the first employee earning more than 100000 (short-circuiting).

```
employees.stream()
        .filter(e -> e.getSalary() > 100000)
        .findFirst()
        .map(Employee::getName)
        .orElse("None");
```

Output: `Rohit Verma`

Q69. Find employees with more than 10 years of experience (assume current year 2026).

```
int currentYear = 2026;
employees.stream()
        .filter(e -> currentYear - e.getYearOfJoining() > 10)
        .map(e -> e.getName() + " - " + (currentYear - e.getYearOfJoining()) + " yrs")
        .collect(Collectors.toList());
```

Output: `[Priya Sharma - 11 yrs, Rohit Verma - 14 yrs, Neha Gupta - 12 yrs, Vikram Rao - 16 yrs, Suresh Patil - 18 yrs, Arjun Reddy - 13 yrs, Fatima Sheikh - 15 yrs]`

Q70. Group employee names by the first letter of their name.

```
employees.stream()
        .collect(Collectors.groupingBy(e -> e.getName().charAt(0),
                TreeMap::new,
                Collectors.mapping(Employee::getName, Collectors.toList())));
```

Output:
```
{A=[Aarav Mehta, Anjali Nair, Arjun Reddy], D=[Divya Menon],
 F=[Fatima Sheikh], I=[Imran Khan], K=[Karan Singh],
 M=[Meera Iyer, Manish Joshi], N=[Neha Gupta], P=[Priya Sharma],
 R=[Rohit Verma], S=[Sneha Kulkarni, Suresh Patil], V=[Vikram Rao]}
```

Q71. Collect names and salaries into a LinkedHashMap ordered by salary descending.

```
employees.stream()
        .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
        .collect(Collectors.toMap(Employee::getName, Employee::getSalary,
                (a, b) -> a, LinkedHashMap::new));
```

Output: `{Vikram Rao=140000.0, Rohit Verma=125000.0, Fatima Sheikh=118000.0, Arjun Reddy=110000.0, Suresh Patil=99000.0, ... Manish Joshi=40000.0}`

Q72. Find names of employees starting with 'A' or 'S'.

```
employees.stream()
        .map(Employee::getName)
        .filter(n -> n.startsWith("A") || n.startsWith("S"))
        .collect(Collectors.toList());
```

Output: `[Aarav Mehta, Sneha Kulkarni, Anjali Nair, Suresh Patil, Arjun Reddy]`

Q73. Find the average age of employees in the IT department.

```
employees.stream()
        .filter(e -> "IT".equals(e.getDepartment()))
        .collect(Collectors.averagingInt(Employee::getAge));
```

Output: 32.4

Q74. Print the employee count per department, sorted by count descending then department name.

```
employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment, Collectors.counting()))
        .entrySet().stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed()
                .thenComparing(Map.Entry::getKey))
        .forEach(e -> System.out.println(e.getKey() + " = " + e.getValue()));
```

Output:
```
IT = 5
Finance = 3
Sales = 3
HR = 2
Marketing = 2
```

Q75. Using reduce, find the employee with the highest salary (without max()).

```
employees.stream()
        .reduce((a, b) -> a.getSalary() >= b.getSalary() ? a : b)
        .map(Employee::getName)
        .orElse("N/A");
```

Output: Vikram Rao

Appendix — Collectors cheat sheet

Collector	What it does
`Collectors.toList() / toSet()`	collect elements into a List or Set
`Collectors.toMap(k, v)`	build a Map; throws on duplicate keys unless a merge function is given
`Collectors.toMap(k, v, merge, LinkedHashMap::new)`	Map with duplicate handling and a chosen implementation
`Collectors.joining(", ", "[", "]")`	concatenate Strings with delimiter, prefix and suffix
`Collectors.counting()`	long count, used as a downstream collector
`Collectors.summingDouble / summingInt`	total of a numeric field
`Collectors.averagingDouble / averagingInt`	average of a numeric field, returns Double
`Collectors.summarizingDouble`	count, sum, min, average and max in one pass
`Collectors.minBy / maxBy(cmp)`	Optional of the smallest or largest element
`Collectors.groupingBy(fn)`	Map<key, List<T>>
`Collectors.groupingBy(fn, downstream)`	group then reduce each group
`Collectors.groupingBy(fn, TreeMap::new, downstream)`	group into a sorted Map
`Collectors.partitioningBy(pred)`	Map<Boolean, List<T>>, always has both true and false keys
`Collectors.mapping(fn, downstream)`	transform elements before the downstream collector
`Collectors.flatMapping(fn, downstream)`	flatten nested collections inside a group (Java 9+)
`Collectors.collectingAndThen(c, finisher)`	post-process a collector result, e.g. unwrap an Optional
`Collectors.teeing(c1, c2, merger)`	run two collectors and combine both results (Java 12+)

Things interviewers like to probe

- Streams are lazy: nothing runs until a terminal operation such as collect, forEach, count, reduce or findFirst is called.
- A stream can be consumed only once. Reusing one throws IllegalStateException: stream has already been operated upon or closed.
- findFirst versus findAny: identical on a sequential stream, but findAny may return any element on a parallel stream.
- Collectors.toMap throws IllegalStateException on a duplicate key — always pass a merge function when keys can repeat (department, city, gender).
- map versus flatMap: map is one-to-one, flatMap flattens a stream of collections into a single stream (used here for skills).
- Arrays.asList returns a fixed-size list backed by an array, so add and remove throw UnsupportedOperationException. Prefer List.of for immutability or new ArrayList<>(...) when you need to modify.
- peek is for debugging only; it may be skipped entirely if the pipeline can be optimised away.
- Prefer mapToDouble / mapToInt over map when summing — it avoids boxing and gives you summaryStatistics.